

✓ Análisis de sentimiento y minería de opiniones turísticas en reseñas de TripAdvisor en español

Trabajo final de la asignatura Minería de datos y análisis de sentimiento: usos y aplicaciones en el sector turístico

Este trabajo tiene como objetivo aplicar técnicas básicas de minería de texto y análisis de sentimiento a un conjunto de reseñas turísticas en español. Para ello, se utilizarán datos procedentes de reseñas de hoteles, restaurantes y museos, con el fin de comparar la percepción de los usuarios en distintos tipos de experiencias turísticas.

El análisis se desarrollará mediante Python en un Jupyter Notebook, combinando técnicas de limpieza textual, exploración de datos, análisis de frecuencias, visualización y análisis de sentimiento.

Índice

1. Introducción
2. Carga y exploración de datos
3. Detección de idioma
4. Limpieza y preprocesamiento textual
5. Análisis descriptivo
6. Análisis léxico
7. Bigramas
8. Nubes de palabras
9. Análisis de sentimiento
10. Conclusiones

1. Introducción

Las reseñas publicadas por usuarios en plataformas turísticas constituyen una fuente de información muy valiosa para analizar la percepción de los visitantes. Estos textos permiten identificar aspectos positivos y negativos de la experiencia turística, como la calidad del servicio, la limpieza, la ubicación, la comida, el precio o el interés cultural de una visita.

Desde la perspectiva de la minería de textos, estas opiniones pueden estudiarse mediante técnicas de procesamiento del lenguaje natural, análisis estadístico y visualización de datos. En este trabajo se parte de varios corpus de reseñas turísticas en español y se aplican procedimientos básicos de limpieza, análisis léxico y análisis de sentimiento.

El objetivo no es únicamente obtener gráficos o métricas, sino interpretar qué patrones lingüísticos y valorativos aparecen en las reseñas y cómo varían según el tipo de servicio turístico analizado.

✓ 2. Objetivos

Los objetivos principales de este trabajo son los siguientes:

1. Cargar y explorar diferentes conjuntos de reseñas turísticas en español.
2. Aplicar técnicas de limpieza y normalización textual mediante Python y expresiones regulares.
3. Analizar las características generales de las reseñas, como su longitud y distribución.
4. Identificar las palabras y expresiones más frecuentes en cada tipo de corpus.
5. Comparar reseñas de hoteles, restaurantes y museos.
6. Realizar un análisis básico de sentimiento para observar la polaridad de las opiniones.
7. Interpretar los resultados obtenidos desde una perspectiva turística y lingüística.

```
# Importamos las librerías principales que se utilizarán en el análisis

import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt

from collections import Counter
```

En esta primera celda se importan las librerías necesarias para el análisis.

La librería `pandas` permite cargar y manipular datos en formato tabular. `numpy` se utiliza para operaciones numéricas básicas. La librería `re` permite trabajar con expresiones regulares, que resultan útiles para limpiar y normalizar texto. `matplotlib` se empleará para generar gráficos, y `Counter` servirá para calcular frecuencias de palabras.

```
from google.colab import files

uploaded = files.upload()
```

Elegir archivos 3 archivos

reviews_museos_es.csv(text/csv) - 86227 bytes, last modified: 18/5/2026 - 100% done
tripadvisor_hoteles_es.csv(text/csv) - 76667 bytes, last modified: 18/5/2026 - 100% done
tripadvisor_restaurantes_es.csv(text/csv) - 74635 bytes, last modified: 18/5/2026 - 100% done
Saving reviews_museos_es.csv to reviews_museos_es.csv
Saving tripadvisor_hoteles_es.csv to tripadvisor_hoteles_es.csv
Saving tripadvisor_restaurantes_es.csv to tripadvisor_restaurantes_es.csv

Esta celda permite subir al entorno de trabajo los archivos CSV que contienen las reseñas turísticas. En este caso, se utilizarán los datasets de hoteles, restaurantes y museos. El objetivo es trabajar con datos reales y comparables entre sí.

```
hoteles = pd.read_csv("tripadvisor_hoteles_es.csv", sep='\t')
restaurantes = pd.read_csv("tripadvisor_restaurantes_es.csv", sep='\t')
museos = pd.read_csv("reviews_museos_es.csv", sep='\t')
```

```
print("Hoteles")
display(hoteles.head())

print("Restaurantes")
display(restaurantes.head())

print("Museos")
display(museos.head())
```

Hoteles

	text	genre	eval_entity_type	name	province	source	url	date	score	
0	La zona del hotel es bonita y está a un paseo ...	review	urban	Senator Granada Spa Hotel	GR	Tripadvisor	https://www.tripadvisor.es/Hotel_Review-g18744...	2017-5-2	4	
1	Tuve la suerte de compartir con este hotel un ...	review	urban	Hotel Palacio de Ubeda	JA	Tripadvisor	https://www.tripadvisor.es/Hotel_Review-g58027...	2018-10-2	5	
2	Llegamos después de un largo viaje en moto a d...	review	urban	OYO Hotel Boutique Reina Mora	GR	Tripadvisor	https://www.tripadvisor.es/Hotel_Review-g18744...	2017-2-2	5	
3	Es la tercera vez que nos alojamos en este hot...	review	beach	Helios Costa Tropical	GR	Tripadvisor	https://www.tripadvisor.es/Hotel_Review-g60898...	2020-8-2	5	
4	Teniendo en cuenta el precio, el hotel cumple ...	review	urban	Zeus Hotel Malaga	MA	Tripadvisor	https://www.tripadvisor.es/Hotel_Review-g18743...	2019-9-2	4	

Restaurantes

	text	genre	eval_entity_type	name	province	source	url	date	score	
--	------	-------	------------------	------	----------	--------	-----	------	-------	--

Antes de iniciar el análisis, es necesario observar la estructura de los datos. La función `head()` permite visualizar las primeras filas de cada dataset y comprobar qué columnas contiene cada archivo. Este paso es importante para identificar la columna que contiene el texto de las reseñas y posibles variables adicionales, como puntuaciones, fechas o nombres de establecimientos.

```
print("Información del dataset de hoteles:")
hoteles.info()

print("\nInformación del dataset de restaurantes:")
restaurantes.info()
```

```
print("\nInformación del dataset de museos:")
museos.info()
```

```
Información del dataset de hoteles:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   text                   100 non-null   object
1   genre                  100 non-null   object
2   eval_entity_type       100 non-null   object
3   name                   100 non-null   object
4   province               100 non-null   object
5   source                 100 non-null   object
6   url                    100 non-null   object
7   date                   100 non-null   object
8   score                  100 non-null   int64
dtypes: int64(1), object(8)
memory usage: 7.2+ KB
```

```
Información del dataset de restaurantes:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   text                   100 non-null   object
1   genre                  100 non-null   object
2   eval_entity_type       100 non-null   object
3   name                   100 non-null   object
4   province               100 non-null   object
5   source                 100 non-null   object
6   url                    100 non-null   object
7   date                   100 non-null   object
8   score                  100 non-null   int64
dtypes: int64(1), object(8)
memory usage: 7.2+ KB
```

```
Información del dataset de museos:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   text                   150 non-null   object
1   Museo                  150 non-null   object
2   Estrellas              150 non-null   int64
3   Fecha de la visita     150 non-null   object
4   Fecha de la opinión    150 non-null   object
5   Enlace                 150 non-null   object
dtypes: int64(1), object(5)
memory usage: 7.2+ KB
```

Con `info()` se revisa el número de filas, columnas, tipos de datos y valores nulos de cada dataset. Esta información es fundamental para decidir si es necesario eliminar valores vacíos, transformar columnas o adaptar el análisis a la estructura concreta de los datos.

```
print("Número de reseñas de hoteles:", len(hoteles))
print("Número de reseñas de restaurantes:", len(restaurantes))
print("Número de reseñas de museos:", len(museos))
```

```
Número de reseñas de hoteles: 100
Número de reseñas de restaurantes: 100
Número de reseñas de museos: 150
```

En esta celda se calcula el número total de reseñas disponibles en cada corpus. Este dato permite conocer el tamaño de cada muestra y valorar si los tres conjuntos son comparables o si alguno de ellos tiene un volumen de datos mucho mayor que los demás.

```
print("Columnas hoteles:", hoteles.columns.tolist())
print("Columnas restaurantes:", restaurantes.columns.tolist())
print("Columnas museos:", museos.columns.tolist())
```

```
Columnas hoteles: ['text', 'genre', 'eval_entity_type', 'name', 'province', 'source', 'url', 'date', 'score']
Columnas restaurantes: ['text', 'genre', 'eval_entity_type', 'name', 'province', 'source', 'url', 'date', 'score']
Columnas museos: ['text', 'Museo', 'Estrellas', 'Fecha de la visita', 'Fecha de la opinión', 'Enlace']
```

Esta celda muestra el nombre de las columnas de cada dataset. El objetivo es identificar la columna que contiene el texto de las reseñas, ya que será la variable principal para el análisis textual.

3. Exploración cualitativa inicial de las reseñas

Antes de realizar el preprocesamiento, resulta necesario inspeccionar manualmente algunas reseñas para identificar posibles problemas textuales. Este paso permite detectar elementos como errores ortográficos, signos de puntuación inconsistentes, caracteres especiales, repeticiones, etiquetas HTML o formatos no homogéneos.

Además, esta revisión preliminar permite observar diferencias estilísticas entre los distintos tipos de reseñas turísticas.

```
# Mostramos algunas reseñas de ejemplo

print("RESEÑAS HOTELES")
for i in range(3):
    print("\n", hoteles["text"].iloc[i])

print("\n" + "="*80)

print("\nRESEÑAS RESTAURANTES")
for i in range(3):
    print("\n", restaurantes["text"].iloc[i])

print("\n" + "="*80)

print("\nRESEÑAS MUSEOS")
for i in range(3):
    print("\n", museos["text"].iloc[i])
```

RESEÑAS HOTELES

La zona del hotel es bonita y está a un paseo del centro de la ciudad. Con la reserva entraba una sesión de Spa, el cual es
 Tuve la suerte de compartir con este hotel un momento tan especial como mi boda . Habló en nombre mío y de mi pareja. El tr
 Llegamos después de un largo viaje en moto a dar una cálida bienvenida y informativo. Las habitaciones estaban limpias, cor
 =====

RESEÑAS RESTAURANTES

I met this restaurant this summer and I just love it! They serve foods from different countries with a especial flavour mac
 Had an amazing meal at El Viajero Del Merkao. Saw the trip advisor and Yelp reviews were very good so tried it and was extr
 Four of us enjoyed a very pleasant meal here last Saturday evening. We had a variety of pasta meals and all greatly enjoyec
 =====

RESEÑAS MUSEOS

Hijo renombrado de Malaga. Ver cómo tuvo sus diferentes etapas como pintor, Picasso artista controversial, su vida y obra e
 Lugar encantador. Muy bien equipado con la vida y obra de Picasso. Con vistas a la plaza de la Merced. Buenas instalciones.
 Me esperaba más. No son obras de Picasso muy relevantes; me esperaba más. Por lo demás bien. El edificio es un palacio y me

La exploración manual permite identificar algunas características frecuentes en este tipo de corpus, como la presencia de lenguaje informal, estructuras expresivas, puntuación enfática o diferencias en la extensión de las reseñas.

También permite detectar posibles elementos que deberán tratarse durante la fase de limpieza y normalización textual.

```
# Comprobamos si existen valores nulos

print("Valores nulos hoteles:", hoteles["text"].isnull().sum())
print("Valores nulos restaurantes:", restaurantes["text"].isnull().sum())
print("Valores nulos museos:", museos["text"].isnull().sum())
```

```
Valores nulos hoteles: 0
Valores nulos restaurantes: 0
Valores nulos museos: 0
```

También se verifica la existencia de valores nulos en la columna textual. Este control es importante porque los valores vacíos pueden provocar errores durante determinadas operaciones de procesamiento lingüístico.

4. Filtrado lingüístico del corpus

Durante la exploración manual de las reseñas se observó la presencia de algunos textos escritos en inglés, especialmente en el corpus de restaurantes.

Dado que el objetivo del trabajo es analizar reseñas turísticas en español, se realizará un filtrado lingüístico básico para conservar únicamente los textos en español y garantizar una mayor coherencia en el análisis posterior.

```
!pip install langdetect
```

```
Collecting langdetect
  Downloading langdetect-1.0.9.tar.gz (981 kB)
    |#####| 981.5/981.5 kB 18.6 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from langdetect) (1.17.0)
Building wheels for collected packages: langdetect
  Building wheel for langdetect (setup.py) ... done
  Created wheel for langdetect: filename=langdetect-1.0.9-py3-none-any.whl size=993223 sha256=b5990bd2cca896ac845596c0ac0c8e
  Stored in directory: /root/.cache/pip/wheels/c1/67/88/e844b5b022812e15a52e4eaa38a1e709e99f06f6639d7e3ba7
Successfully built langdetect
Installing collected packages: langdetect
Successfully installed langdetect-1.0.9
```

```
from langdetect import detect
```

```
# Función para detectar idioma
```

```
def detectar_idioma(texto):
    try:
        return detect(texto)
    except:
        return "unknown"
```

```
# Detectamos idioma de las reseñas
```

```
hoteles["language"] = hoteles["text"].apply(detectar_idioma)
restaurantes["language"] = restaurantes["text"].apply(detectar_idioma)
museos["language"] = museos["text"].apply(detectar_idioma)
```

```
print(hoteles["language"].value_counts())

print("\n" + "="*50)

print(restaurantes["language"].value_counts())

print("\n" + "="*50)

print(museos["language"].value_counts())
```

```
language
es    100
Name: count, dtype: int64
```

```
=====
language
en    100
Name: count, dtype: int64
```

```
=====
language
es    150
Name: count, dtype: int64
```

La detección automática de idioma permite identificar la presencia de textos escritos en lenguas distintas del español. Este paso es importante para evitar que el análisis léxico y el análisis de sentimiento se vean afectados por la mezcla de idiomas dentro del corpus.

```
# Conservamos todos los corpus
# Se observa que el corpus de restaurantes contiene principalmente reseñas en inglés
```

```
print(hoteles["language"].value_counts())

print("\n" + "="*50)

print(restaurantes["language"].value_counts())

print("\n" + "="*50)

print(museos["language"].value_counts())
```

```
language
es    100
Name: count, dtype: int64
```

```
=====
language
en    100
Name: count, dtype: int64
```

```
=====
language
es      150
Name: count, dtype: int64
```

La detección de idioma muestra una distribución lingüística claramente diferenciada entre los corpus. Mientras que las reseñas de hoteles y museos están escritas principalmente en español, el corpus de restaurantes contiene mayoritariamente textos en inglés.

Esta diferencia se conservará en el análisis con fines comparativos y se tendrá en cuenta durante la interpretación de los resultados.

5. Limpieza y preprocesamiento textual

El preprocesamiento constituye una fase fundamental dentro de cualquier proyecto de minería de texto. Los corpus reales suelen contener inconsistencias ortográficas, espacios innecesarios, signos de puntuación irregulares o elementos que dificultan el análisis automático.

En esta fase se aplicarán diferentes técnicas de limpieza y normalización utilizando Python y expresiones regulares. El objetivo es obtener versiones más homogéneas de las reseñas sin alterar su contenido semántico principal.

```
# Creamos copias de trabajo de los datasets

hoteles_clean = hoteles.copy()
restaurantes_clean = restaurantes.copy()
museos_clean = museos.copy()
```

Antes de iniciar el preprocesamiento, se crean copias independientes de los datasets originales. De este modo, los datos originales permanecen intactos y todas las transformaciones se realizan sobre nuevas versiones de trabajo.

```
# Convertimos el texto a minúsculas

hoteles_clean["clean_text"] = hoteles_clean["text"].str.lower()
restaurantes_clean["clean_text"] = restaurantes_clean["text"].str.lower()
museos_clean["clean_text"] = museos_clean["text"].str.lower()
```

La conversión a minúsculas permite evitar duplicidades léxicas durante el análisis. Por ejemplo, las palabras “Hotel” y “hotel” pasarán a considerarse la misma unidad textual.

```
# Eliminamos espacios innecesarios

hoteles_clean["clean_text"] = hoteles_clean["clean_text"].str.strip()
restaurantes_clean["clean_text"] = restaurantes_clean["clean_text"].str.strip()
museos_clean["clean_text"] = museos_clean["clean_text"].str.strip()
```

```
# Eliminamos URLs mediante expresiones regulares

pattern_url = r"http\S+|www\S+"

hoteles_clean["clean_text"] = hoteles_clean["clean_text"].str.replace(pattern_url, "", regex=True)
restaurantes_clean["clean_text"] = restaurantes_clean["clean_text"].str.replace(pattern_url, "", regex=True)
museos_clean["clean_text"] = museos_clean["clean_text"].str.replace(pattern_url, "", regex=True)
```

También se eliminan posibles direcciones web o URLs mediante expresiones regulares. Estos elementos no aportan información relevante para el análisis lingüístico y pueden introducir ruido en los resultados.

```
# Reducimos caracteres repetidos exagerados

def reducir_repeticiones(texto):
    return re.sub(r'(\.)\1{2,}', r'\1', texto)

hoteles_clean["clean_text"] = hoteles_clean["clean_text"].apply(reducir_repeticiones)
restaurantes_clean["clean_text"] = restaurantes_clean["clean_text"].apply(reducir_repeticiones)
museos_clean["clean_text"] = museos_clean["clean_text"].apply(reducir_repeticiones)
```

En las reseñas online aparecen frecuentemente secuencias de caracteres repetidos con finalidad expresiva, como “buenoooo” o “geniaaaal”. Mediante expresiones regulares se reducen estas repeticiones excesivas para homogeneizar el corpus sin eliminar completamente la carga expresiva del texto.

```
# Eliminamos parte de la puntuación
# Conservamos exclamaciones e interrogaciones por su posible valor emocional
```

```
pattern_puntuacion = r"\"#$%&'()*+,-/:;=>@[\\]^_`{|}~]"

hoteles_clean["clean_text"] = hoteles_clean["clean_text"].str.replace(pattern_puntuacion, " ", regex=True)
restaurantes_clean["clean_text"] = restaurantes_clean["clean_text"].str.replace(pattern_puntuacion, " ", regex=True)
museos_clean["clean_text"] = museos_clean["clean_text"].str.replace(pattern_puntuacion, " ", regex=True)
```

No toda la puntuación posee el mismo valor lingüístico. En este trabajo se eliminan signos secundarios, pero se conservan exclamaciones e interrogaciones debido a su posible función expresiva dentro de las reseñas.

```
# Eliminamos espacios múltiples

hoteles_clean["clean_text"] = hoteles_clean["clean_text"].str.replace(r"\s+", " ", regex=True)
restaurantes_clean["clean_text"] = restaurantes_clean["clean_text"].str.replace(r"\s+", " ", regex=True)
museos_clean["clean_text"] = museos_clean["clean_text"].str.replace(r"\s+", " ", regex=True)
```

```
# Comparación entre texto original y texto limpio
```

```
print("TEXTO ORIGINAL:\n")
print(hoteles["text"].iloc[0])

print("\n" + "="*80)

print("\nTEXTO LIMPIO:\n")
print(hoteles_clean["clean_text"].iloc[0])
```

```
1 centro de la ciudad. Con la reserva entraba una sesión de Spa, el cual es bastante grande. También cogimos el desayuno que
=====
```

```
1 centro de la ciudad. con la reserva entraba una sesión de spa, el cual es bastante grande. también cogimos el desayuno que
```

La comparación entre el texto original y la versión preprocesada permite observar las transformaciones realizadas durante la limpieza textual y comprobar que el contenido principal de las reseñas se mantiene intacto.

6. Análisis descriptivo del corpus

Una vez realizado el preprocesamiento, se procede a realizar un análisis descriptivo de los corpus. El objetivo es explorar algunas características generales de las reseñas, como su longitud, distribución y diferencias entre tipos de experiencias turísticas.

Este tipo de análisis permite identificar patrones básicos antes de aplicar técnicas más avanzadas de minería textual y análisis de sentimiento.

```
# Calculamos la longitud de las reseñas en número de palabras

hoteles_clean["word_count"] = hoteles_clean["clean_text"].apply(lambda x: len(x.split()))
restaurantes_clean["word_count"] = restaurantes_clean["clean_text"].apply(lambda x: len(x.split()))
museos_clean["word_count"] = museos_clean["clean_text"].apply(lambda x: len(x.split()))
```

En esta fase se calcula la longitud de cada reseña en número de palabras. Esta variable permite comparar el grado de detalle y extensión de las opiniones publicadas por los usuarios en los distintos corpus turísticos.

```
print("MEDIA DE PALABRAS POR RESEÑA\n")

print("Hoteles:", round(hoteles_clean["word_count"].mean(), 2))
print("Restaurantes:", round(restaurantes_clean["word_count"].mean(), 2))
print("Museos:", round(museos_clean["word_count"].mean(), 2))
```

```
MEDIA DE PALABRAS POR RESEÑA
```

```
Hoteles: 96.08
Restaurantes: 98.72
Museos: 62.29
```

La media de palabras por reseña permite observar diferencias en la extensión de los comentarios según el tipo de experiencia turística. Las reseñas más largas suelen contener descripciones más detalladas, mientras que las más breves tienden a ser evaluaciones más directas.

```
# Gráfico comparativo de longitud media

categorias = ["Hoteles", "Restaurantes", "Museos"]

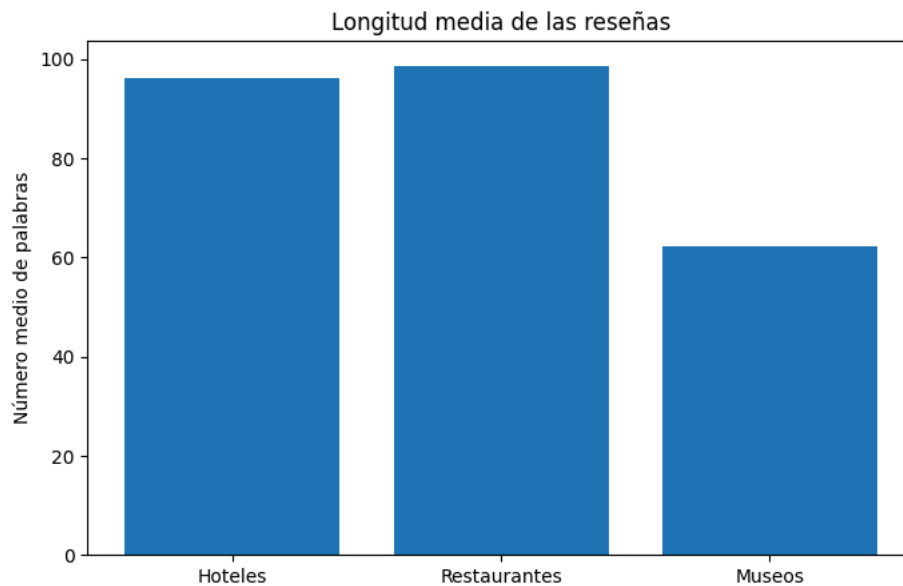
medias = [
    hoteles_clean["word_count"].mean(),
    restaurantes_clean["word_count"].mean(),
    museos_clean["word_count"].mean()
]

plt.figure(figsize=(8,5))

plt.bar(categorias, medias)

plt.ylabel("Número medio de palabras")
plt.title("Longitud media de las reseñas")

plt.show()
```



El gráfico permite comparar visualmente la longitud media de las reseñas en los distintos corpus. Estas diferencias pueden reflejar distintos estilos discursivos y diferentes formas de evaluar la experiencia turística.

```
# Distribución de longitud de reseñas

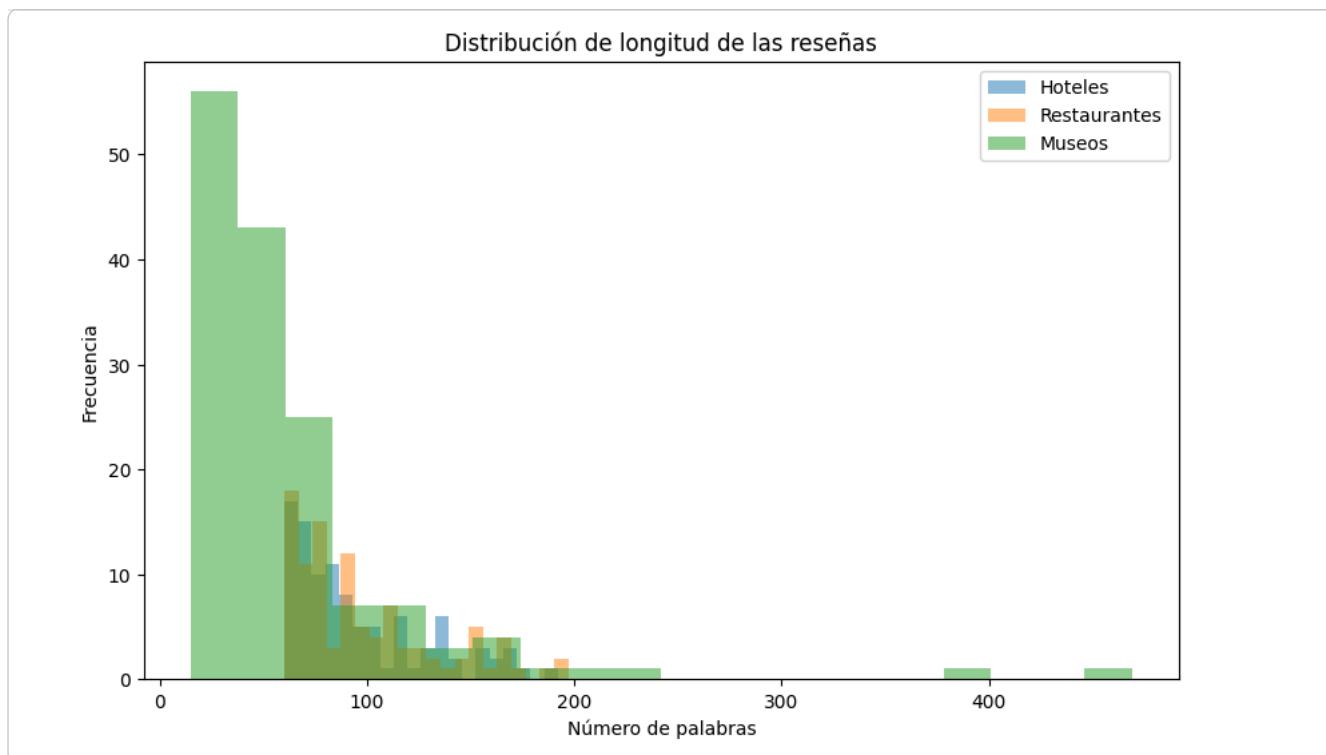
plt.figure(figsize=(10,6))

plt.hist(hoteles_clean["word_count"], bins=20, alpha=0.5, label="Hoteles")
plt.hist(restaurantes_clean["word_count"], bins=20, alpha=0.5, label="Restaurantes")
plt.hist(museos_clean["word_count"], bins=20, alpha=0.5, label="Museos")

plt.xlabel("Número de palabras")
plt.ylabel("Frecuencia")
plt.title("Distribución de longitud de las reseñas")

plt.legend()

plt.show()
```

La distribución de frecuencias permite observar no solo la media, sino también la variabilidad en la longitud de las reseñas. Algunos corpus presentan comentarios más homogéneos, mientras que otros muestran una mayor dispersión en el tamaño de los textos.

Los resultados muestran diferencias relevantes entre los distintos corpus turísticos. Las reseñas de restaurantes presentan, en promedio, una mayor longitud, lo que puede relacionarse con un estilo discursivo más narrativo y experiencial. Los usuarios suelen describir aspectos como la comida, el servicio, el ambiente o la relación calidad-precio con bastante detalle.

Por el contrario, las reseñas de museos tienden a ser más breves y directas, aunque el histograma revela también la existencia de algunos comentarios excepcionalmente extensos. Esto sugiere la coexistencia de dos patrones discursivos diferentes: evaluaciones rápidas y descripciones culturales más elaboradas.

Las reseñas hoteleras presentan un comportamiento intermedio entre ambos corpus.

7. Análisis léxico del corpus

El análisis léxico permite identificar las palabras más frecuentes dentro de cada corpus y observar qué conceptos aparecen con mayor relevancia en las reseñas.

Este tipo de análisis resulta útil para detectar temas recurrentes, preferencias de los usuarios y diferencias discursivas entre distintos tipos de experiencias turísticas.

```
import nltk
nltk.download('stopwords')

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
True
```

```
from nltk.corpus import stopwords
```

```
stopwords_es = set(stopwords.words('spanish'))
stopwords_en = set(stopwords.words('english'))
```

```
# Función para tokenizar y limpiar palabras

def obtener_palabras(textos, stopwords_set):

    palabras = []

    for texto in textos:

        tokens = texto.split()

        tokens_limpios = [
```

```

        palabra for palabra in tokens
        if palabra.isalpha() and palabra not in stopwords_set
    ]

    palabras.extend(tokens_limpios)

return palabras

```

```
# Hoteles (español)
```

```
palabras_hoteles = obtener_palabras(
    hoteles_clean["clean_text"],
    stopwords_es
)
```

```
# Restaurantes (inglés)
```

```
palabras_restaurantes = obtener_palabras(
    restaurantes_clean["clean_text"],
    stopwords_en
)
```

```
# Museos (español)
```

```
palabras_museos = obtener_palabras(
    museos_clean["clean_text"],
    stopwords_es
)
```

```
freq_hoteles = Counter(palabras_hoteles)
freq_restaurantes = Counter(palabras_restaurantes)
freq_museos = Counter(palabras_museos)
```

```

print("PALABRAS MÁS FRECUENTES - HOTELES\n")
print(freq_hoteles.most_common(15))

print("\n" + "="*70)

print("\nPALABRAS MÁS FRECUENTES - RESTAURANTES\n")
print(freq_restaurantes.most_common(15))

print("\n" + "="*70)

print("\nPALABRAS MÁS FRECUENTES - MUSEOS\n")
print(freq_museos.most_common(15))

```

```
PALABRAS MÁS FRECUENTES - HOTELES
```

```
[('hotel', 111), ('personal', 49), ('habitación', 46), ('habitaciones', 35), ('desayuno', 30), ('bien', 30), ('si', 27), ('t
```

```
=====
```

```
PALABRAS MÁS FRECUENTES - RESTAURANTES
```

```
[('food', 63), ('restaurant', 43), ('good', 39), ('service', 35), ('great', 29), ('menu', 28), ('place', 27), ('would', 26),
```

```
=====
```

```
PALABRAS MÁS FRECUENTES - MUSEOS
```

```
[('museo', 112), ('visita', 39), ('gran', 36), ('si', 35), ('colección', 34), ('málaga', 33), ('obras', 32), ('ver', 28), ('
```

La extracción de palabras frecuentes permite identificar algunos de los conceptos más recurrentes en las reseñas. Para ello, se eliminan previamente las llamadas *stopwords* o palabras vacías, es decir, términos gramaticales muy frecuentes que aportan poca información semántica.

Dado que el corpus de restaurantes está escrito principalmente en inglés, se utilizan listas de stopwords diferentes según el idioma del corpus.

Los resultados muestran diferencias léxicas claras entre los distintos tipos de corpus turísticos.

En las reseñas hoteleras predominan términos relacionados con la atención al cliente, las habitaciones y los servicios del alojamiento, como “personal”, “habitación”, “desayuno” o “trato”. Esto refleja un enfoque centrado en la comodidad y la calidad del servicio.

En el corpus de restaurantes destacan palabras vinculadas a la experiencia gastronómica, como “food”, “menu”, “wine” o “service”. Además, aparecen numerosos términos valorativos en inglés, como “good” o “great”, lo que sugiere un discurso altamente evaluativo y emocional.

Por su parte, las reseñas de museos presentan un vocabulario claramente cultural y patrimonial, con términos como “colección”, “arte”, “obras” o “exposición”. Esto indica un estilo más descriptivo y orientado al contenido artístico de la experiencia.

```
# Top 10 palabras hoteles

top_hoteles = freq_hoteles.most_common(10)

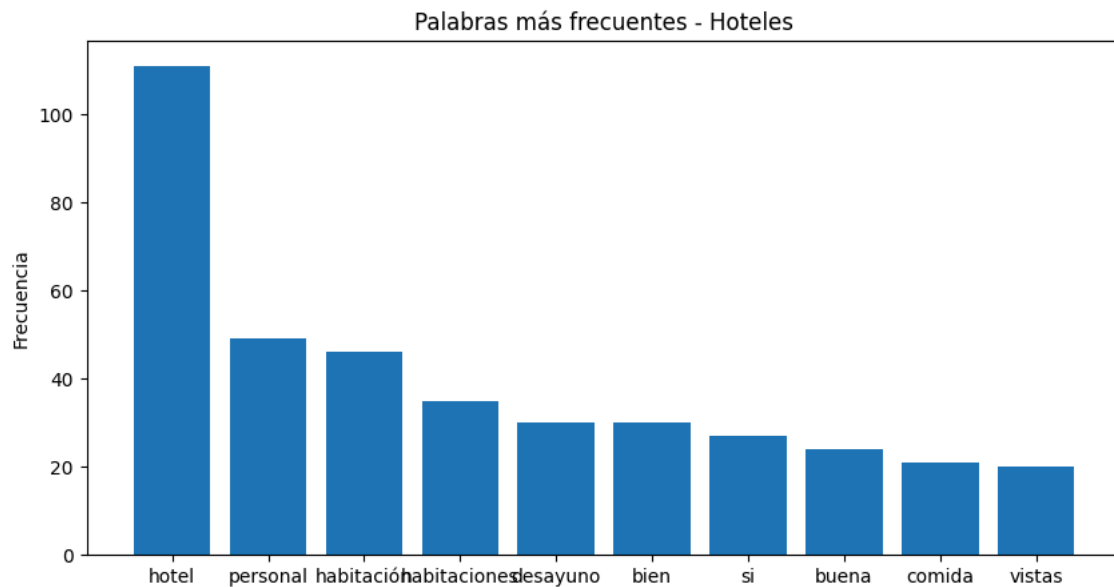
palabras = [x[0] for x in top_hoteles]
frecuencias = [x[1] for x in top_hoteles]

plt.figure(figsize=(10,5))

plt.bar(palabras, frecuencias)

plt.title("Palabras más frecuentes - Hoteles")
plt.ylabel("Frecuencia")

plt.show()
```



```
# Top 10 palabras restaurantes

top_restaurantes = freq_restaurantes.most_common(10)

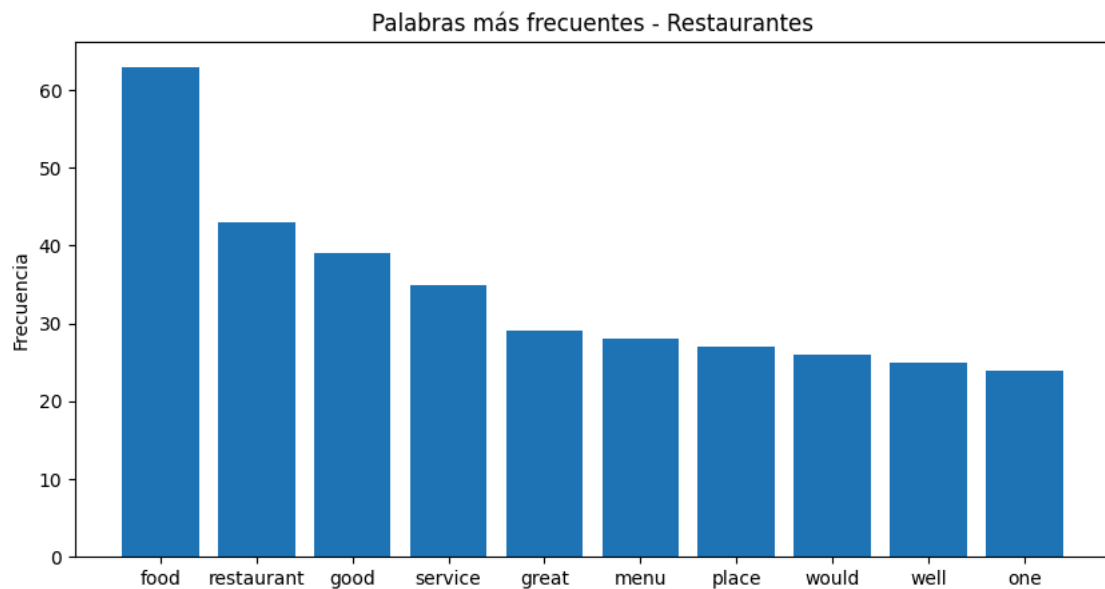
palabras = [x[0] for x in top_restaurantes]
frecuencias = [x[1] for x in top_restaurantes]

plt.figure(figsize=(10,5))

plt.bar(palabras, frecuencias)

plt.title("Palabras más frecuentes - Restaurantes")
plt.ylabel("Frecuencia")

plt.show()
```



```
# Top 10 palabras museos

top_museos = freq_museos.most_common(10)

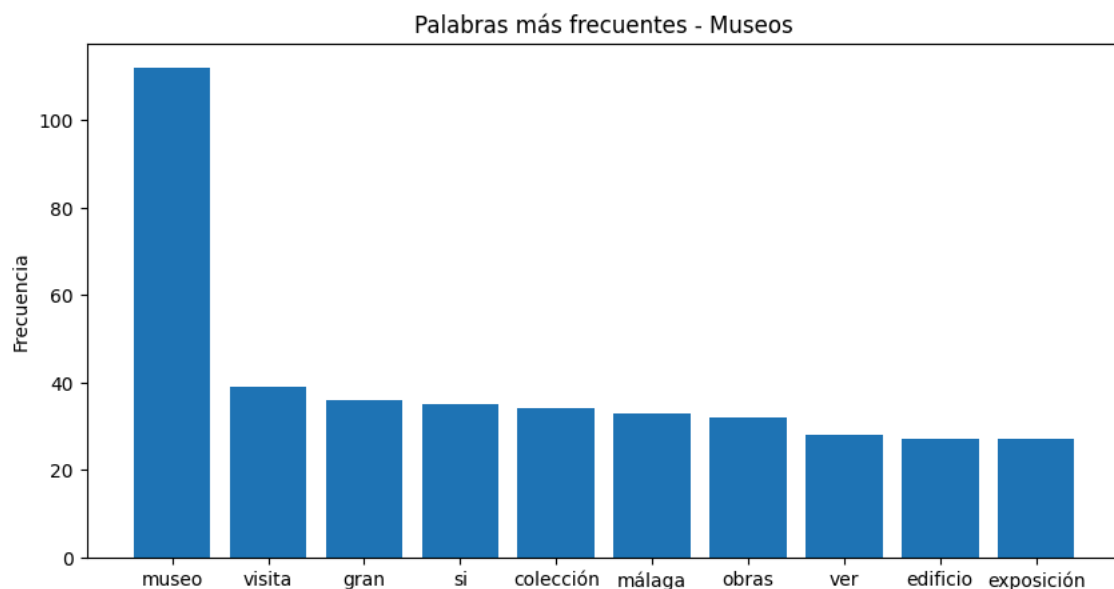
palabras = [x[0] for x in top_museos]
frecuencias = [x[1] for x in top_museos]

plt.figure(figsize=(10,5))

plt.bar(palabras, frecuencias)

plt.title("Palabras más frecuentes - Museos")
plt.ylabel("Frecuencia")

plt.show()
```



La visualización de palabras frecuentes permite observar de forma más intuitiva los conceptos predominantes en cada corpus turístico. Los gráficos muestran diferencias temáticas claras entre alojamiento, gastronomía y turismo cultural.

Los gráficos muestran no solo diferencias temáticas entre los corpus, sino también diferencias en el tipo de lenguaje utilizado por los usuarios.

En las reseñas hoteleras y gastronómicas aparecen numerosos términos valorativos, como “bien”, “buena”, “good” o “great”, lo que refleja una fuerte dimensión evaluativa y emocional. Por el contrario, el corpus de museos contiene un vocabulario más descriptivo y cultural, relacionado con elementos patrimoniales y artísticos.

Esto sugiere que cada tipo de experiencia turística genera patrones discursivos diferentes.

8. Análisis de bigramas

Además de las palabras individuales, resulta útil analizar secuencias frecuentes de dos palabras consecutivas, conocidas como *bigramas*.

Los bigramas permiten identificar expresiones habituales dentro de las reseñas y proporcionan un contexto semántico más rico que las palabras aisladas.

```
from nltk.util import bigrams
```

```
# Función para obtener bigramas
```

```
def obtener_bigramas(lista_palabras):
```

```
    return list(bigrams(lista_palabras))
```

```
bigramas_hoteles = obtener_bigramas(palabras_hoteles)
bigramas_restaurantes = obtener_bigramas(palabras_restaurantes)
bigramas_museos = obtener_bigramas(palabras_museos)
```

```
freq_bigramas_hoteles = Counter(bigramas_hoteles)
freq_bigramas_restaurantes = Counter(bigramas_restaurantes)
freq_bigramas_museos = Counter(bigramas_museos)
```

```
print("BIGRAMAS MÁS FRECUENTES - HOTELES\n")
print(freq_bigramas_hoteles.most_common(10))

print("\n" + "="*70)
```

```
print("\nBIGRAMAS MÁS FRECUENTES - RESTAURANTES\n")
print(freq_bigramas_restaurantes.most_common(10))
```

```
print("\n" + "="*70)
```

```
print("\nBIGRAMAS MÁS FRECUENTES - MUSEOS\n")
print(freq_bigramas_museos.most_common(10))
```

```
BIGRAMAS MÁS FRECUENTES - HOTELES
```

```
[('personal', 'amable'), 7), (('fin', 'semana'), 5), (('personal', 'recepción'), 5), (('habitación', 'vistas'), 5), (('bier
=====
```

```
BIGRAMAS MÁS FRECUENTES - RESTAURANTES
```

```
[('well', 'worth'), 6), (('one', 'best'), 5), (('food', 'excellent'), 5), (('quality', 'food'), 5), (('many', 'restaurants'
=====
```

```
BIGRAMAS MÁS FRECUENTES - MUSEOS
```

```
[('merece', 'pena'), 6), (('gran', 'museo'), 6), (('museo', 'interesante'), 6), (('interesante', 'museo'), 5), (('vale', 'p
```

El análisis de bigramas permite observar combinaciones léxicas frecuentes dentro de las reseñas. Estas expresiones ofrecen información más precisa sobre los aspectos que los usuarios valoran o describen en cada tipo de experiencia turística.

El análisis de bigramas permite identificar patrones fraseológicos característicos de cada tipo de experiencia turística.

En las reseñas hoteleras predominan expresiones relacionadas con el servicio y las instalaciones, como “personal amable”, “aire acondicionado” o “habitaciones amplias”. Esto refleja una preocupación por la comodidad y la calidad funcional del alojamiento.

En el corpus gastronómico aparecen numerosas expresiones valorativas y recomendativas, como “food excellent”, “staff friendly” o “highly recommended”. Este patrón sugiere un discurso más emocional y orientado a la evaluación subjetiva de la experiencia culinaria.

Por su parte, las reseñas de museos presentan combinaciones léxicas claramente vinculadas al ámbito cultural y patrimonial, como “colección permanente”, “exposiciones temporales” o “gran museo”. Estas expresiones muestran un enfoque más descriptivo y cultural.

```
!pip install wordcloud
```

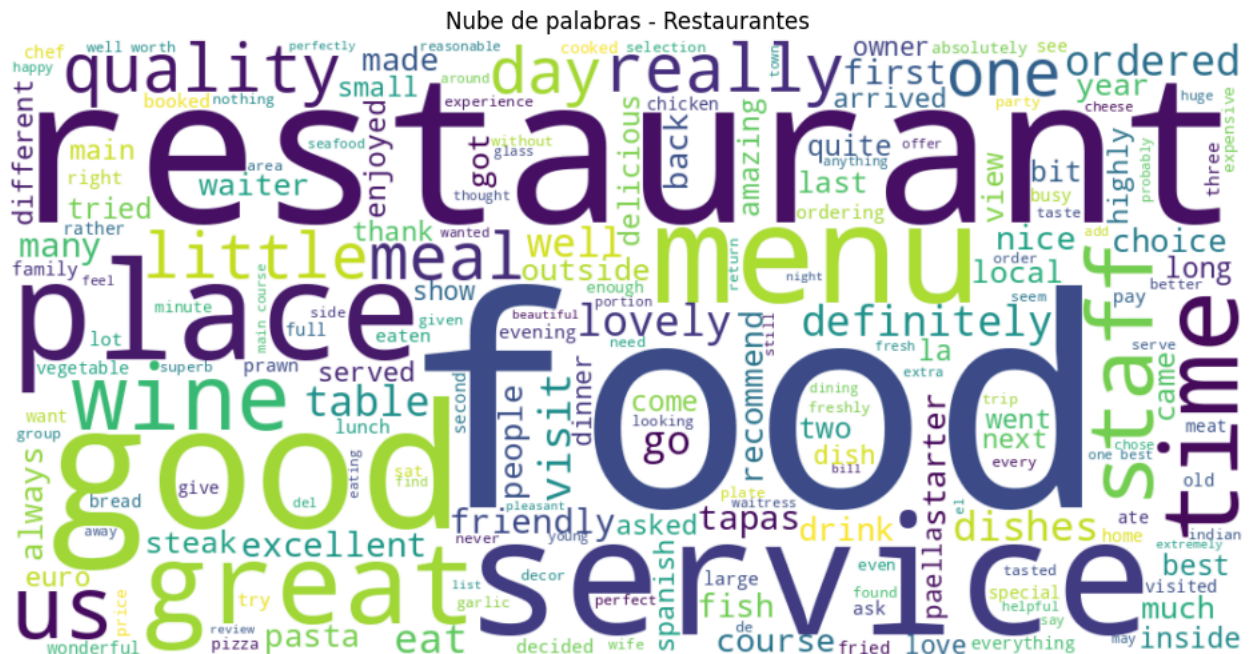
```
Requirement already satisfied: wordcloud in /usr/local/lib/python3.12/dist-packages (1.9.6)
Requirement already satisfied: numpy>=1.19.3 in /usr/local/lib/python3.12/dist-packages (from wordcloud) (2.0.2)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from wordcloud) (11.3.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from wordcloud) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (1.3.0)
```

```
from wordcloud import WordCloud
```

```
plt.show()
```



```
plt.show()
```



Nube de palabras - Museos

```

texto_museos = " ".join(palabras_museos)

wordcloud = WordCloud(
    width=1000,
    height=500,
    background_color='white'
).generate(texto_museos)

plt.figure(figsize=(12,6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.title("Nube de palabras - Museos")

plt.show()

```



Las nubes de palabras permiten representar visualmente los términos más frecuentes de cada corpus. El tamaño de cada palabra está relacionado con su frecuencia de aparición dentro de las reseñas.

Este tipo de visualización facilita la identificación rápida de temas predominantes y patrones léxicos característicos de cada ámbito turístico.

Las nubes de palabras refuerzan los patrones observados previamente en el análisis léxico y de bigramas.

En el corpus hotelero predominan términos relacionados con las instalaciones, la atención al cliente y la comodidad del alojamiento, como “habitación”, “personal”, “desayuno” o “trato”. Esto sugiere una evaluación centrada en aspectos funcionales y de servicio.

Las reseñas gastronómicas muestran un lenguaje claramente evaluativo y emocional, con abundancia de términos como “good”, “great”, “excellent” o “lovely”. El foco principal se sitúa en la experiencia culinaria y en la valoración subjetiva del servicio y la comida.

Por el contrario, el corpus de museos presenta un vocabulario más cultural y patrimonial, con referencias frecuentes a “arte”, “colección”, “historia”, “exposición” o “edificio”. Esto indica un estilo discursivo más descriptivo y orientado al contenido cultural de la experiencia turística.

10. Análisis de sentimiento

El análisis de sentimiento permite estimar automáticamente la polaridad emocional de las reseñas. Este tipo de análisis resulta especialmente útil en el ámbito turístico, ya que permite detectar tendencias generales de satisfacción o insatisfacción de los usuarios.

En este trabajo se utilizará la librería TextBlob para calcular una puntuación aproximada de polaridad en cada reseña.

```
!pip install textblob
```

```
Requirement already satisfied: textblob in /usr/local/lib/python3.12/dist-packages (0.19.0)
Requirement already satisfied: nltk>=3.9 in /usr/local/lib/python3.12/dist-packages (from textblob) (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (8.3.3)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (4.67.3)
```

```
from textblob import TextBlob
```

```
# Función simple de polaridad para textos en inglés
```

```
def obtener_polaridad(texto):

    return TextBlob(texto).sentiment.polarity
```

```
# Calculamos polaridad únicamente para el corpus en inglés
```

```
restaurantes_clean["polarity"] = restaurantes_clean["clean_text"].apply(obtener_polaridad)
```

```
print("POLARIDAD MEDIA - RESTAURANTES\n")

print(
    round(restaurantes_clean["polarity"].mean(), 3)
)
```

```
POLARIDAD MEDIA - RESTAURANTES
```

```
0.267
```

La polaridad oscila entre -1 y 1. Los valores cercanos a 1 indican una valoración positiva, mientras que los valores negativos reflejan opiniones desfavorables.

El cálculo de la polaridad media permite comparar el tono general de las reseñas en los distintos corpus turísticos.

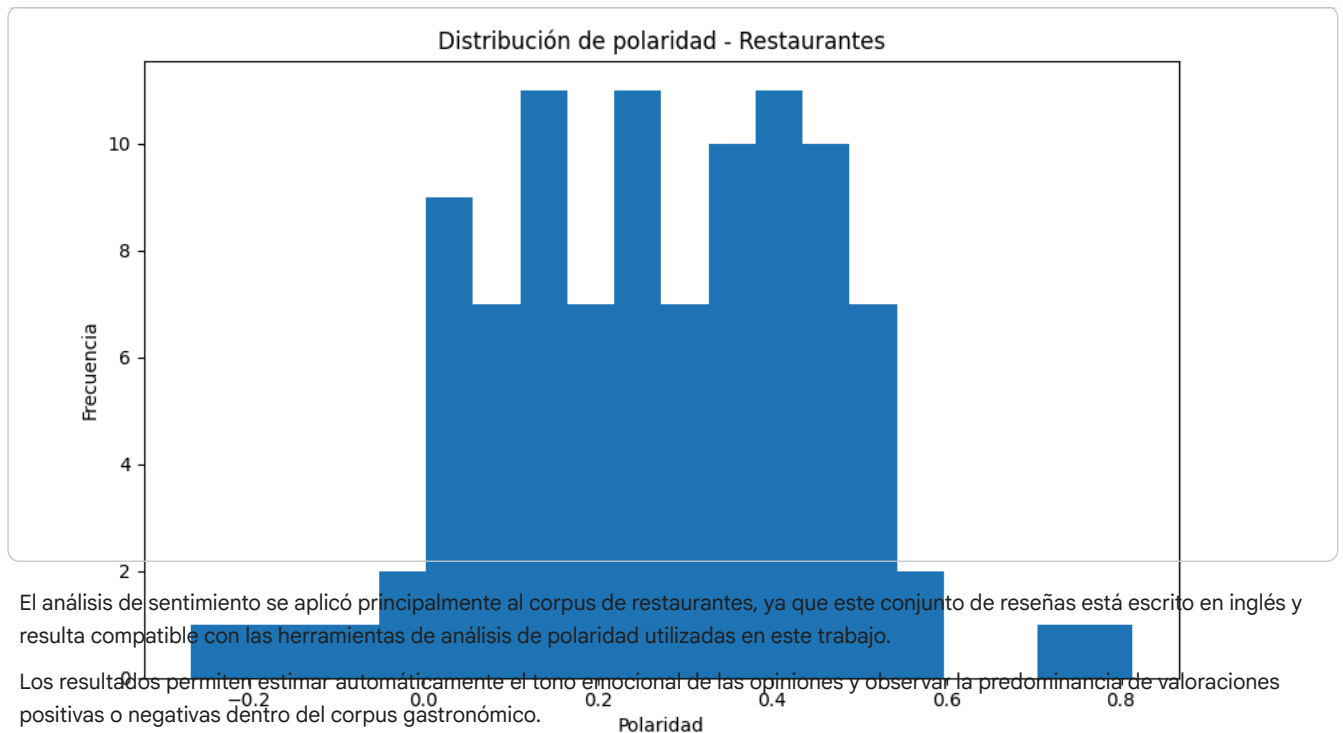
```
plt.figure(figsize=(10,6))

plt.hist(
    restaurantes_clean["polarity"],
    bins=20
)

plt.xlabel("Polaridad")
plt.ylabel("Frecuencia")

plt.title("Distribución de polaridad - Restaurantes")

plt.show()
```

El histograma muestra una clara predominancia de valores positivos en las reseñas gastronómicas, lo que indica una tendencia general de satisfacción por parte de los usuarios.

La mayoría de las opiniones se concentran en polaridades moderadamente positivas, mientras que las valoraciones claramente negativas aparecen con menor frecuencia. Esto sugiere que el corpus contiene principalmente experiencias favorables, aunque existe también cierta diversidad valorativa.

La distribución observada resulta coherente con el lenguaje evaluativo identificado previamente en el análisis léxico y en los bigramas frecuentes.

9.1 Limitaciones metodológicas

Este trabajo presenta algunas limitaciones que deben tenerse en cuenta durante la interpretación de los resultados.

En primer lugar, los corpus utilizados presentan diferencias lingüísticas importantes, ya que las reseñas de restaurantes están escritas principalmente en inglés, mientras que los otros conjuntos de datos se encuentran en español. Esta situación condicionó algunas fases